

Lecture 9--10

Unity C# 프로그래밍

스크립트 기초에서 핵심 기능 체험까지

강영민

동명대학교 게임학부

Part I

Unity 스크립트로서의 C#

일반 C# 프로그램과 Unity 스크립트의 차이

이번 강의에서 다루는 것

학습 목표

- 일반 C# 프로그램과 Unity 스크립트의 **진입 방식**이 어떻게 다른지 이해한다
- MonoBehaviour와 **생명 주기 메서드**의 호출 시점을 파악한다
- Unity의 **컴포넌트 기반 설계**를 이해한다
- Unity의 **Inspector 직렬화**와 노출 규칙을 안다
- Unity 스크립트 작성 시 흔한 **합정과 성능 규칙**을 피할 수 있다

대상

이미 일반 C# 프로그래밍의 기초(클래스, 상속, 메서드, 제어문, 컬렉션 등)는 알고 있는 학생. Unity는 처음 접하더라도, “같은 C# 언어인데 왜 작성 방식이 이렇게 달라 보이는가”에 답을 얻는 것이 목표.

일반 C# 프로그램에서 우리가 직접 했던 일들

⚙️ 콘솔/GUI 프로그램에서 우리가 짰 코드

- **진입점** Main 작성 --- 프로그램이 어디서 시작되는가
- 객체를 new로 직접 생성
- 입력 처리 --- `Console.ReadLine`, 또는 GUI라면 `KeyDown` 이벤트
- 출력 처리 --- `Console.WriteLine`, 또는 GUI라면 `Graphics`로 직접 그리기
- 시간 흐름 처리 --- 타이머 또는 `Thread.Sleep`으로 간격 만들기
- 게임이라면 **게임 루프**를 직접 작성 (위치 갱신 → 충돌 검사 → 다시 그리기)

핵심

Unity에서는 **이 모든 것이 이미 만들어져 있다.**
우리는 `Update()`처럼 **약속된 이름**의 메서드만 채워 넣는다.

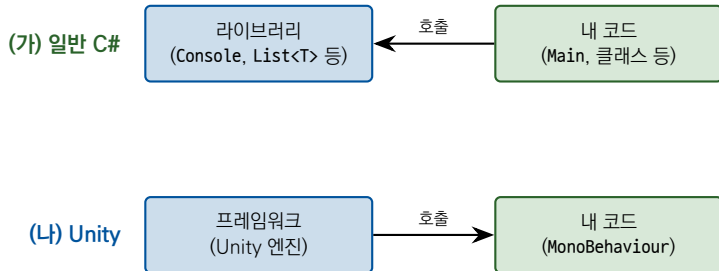
↔ 호출 방향의 “역전”

- **일반 C# 프로그램**: 우리가 라이브러리를 호출한다.
`Console.WriteLine(...), list.Add(...)`
- **Unity 스크립트**: 프레임워크가 우리를 호출한다.
`void Update() { ... }` ← Unity가 매 프레임 자동 호출

헐리우드 원칙 (Hollywood Principle)

이 “역전된 호출 관계”를 **IoC(Inversion of Control)** 또는 **헐리우드 원칙**(“Don’t call us, we’ll call you”)이라 부른다.
프레임워크가 “Update라는 이름의 메서드는 매 프레임 호출하겠다”는 약속을 정해 두면, 우리는 그 약속에 맞춰 메서드를 작성하
기만 하면 된다.

개념도: 라이브러리 vs 프레임워크



💡 한 줄 요약

라이브러리는 “내가 부른다”, 프레임워크는 “나를 부른다”.

가장 기본적인 Unity 스크립트



HelloUnity.cs

```
1 using UnityEngine;
2
3 public class HelloUnity : MonoBehaviour
4 {
5     void Start() { Debug.Log("Hello, Unity!"); }
6     void Update() { /* Called every frame */ }
7 }
```



핵심 4가지

- using UnityEngine; --- Unity 기본 네임스페이스
- : MonoBehaviour --- Unity가 인식하는 "스크립트 컴포넌트"가 되기 위한 필수 상속
- 클래스 이름 = 파일 이름(HelloUnity.cs)이어야 함
- Debug.Log --- Unity Console에 출력 (Console.WriteLine의 Unity 버전)

MonoBehaviour란 정확히 무엇인가?

Unity가 미리 만들어 둔 기반 클래스

- 미리 정의된 “빈” 메서드들 --- 약 30개
Awake, Start, Update, OnCollisionEnter ...
기본 구현은 “아무 것도 안 함”. 우리가 채워 넣는다.
- 유용한 속성들
transform, gameObject, enabled, name ...

주의: override 키워드를 쓰지 않는다

Unity는 메서드 이름을 보고 리플렉션으로 찾아 호출한다. 그래서 override 키워드가 필요 없지만, 반대로 **메서드 이름의 철자가 틀리면 그냥 호출되지 않을 뿐** 컴파일 에러는 나지 않는다. (Updaet, start 같은 실수 주의)

Main이 없다는 게 무슨 뜻인가?

▶ Play 버튼을 누르면 Unity가 하는 일

- 1 현재 **Scene**에 있는 모든 **GameObject**를 찾는다
- 2 각 **GameObject**에 붙어 있는 **MonoBehaviour**를 찾는다
- 3 각 스크립트의 **Awake** → **OnEnable** → **Start**를 차례로 호출
- 4 매 프레임마다 모든 활성 스크립트의 **Update**를 호출

💡 한 줄 요약

Unity 엔진 자체가 거대한 **Main**이라고 생각하면 된다.

우리는 그 안에서 “특정 시점에 실행될 코드”만 약속된 이름의 메서드에 작성한다.

Scene(씬)이란?

☰ “한 화면을 구성하는 GameObject들의 모음”

- TitleScene --- 타이틀 화면 (로고, 시작 버튼)
- StageScene --- 게임 플레이 화면 (플레이어, 적, UI)
- EndingScene --- 엔딩 화면

파일로 저장됨 (.unity 확장자).

`SceneManager.LoadScene("StageScene")`으로 전환.

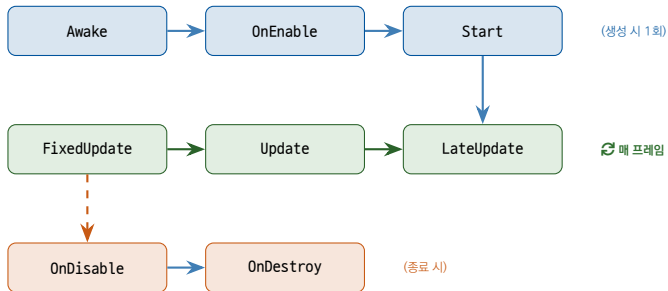
↔ 일반 GUI 프로그램과의 비교

일반 GUI 프로그램에서는 “하나의 폼/창”을 띄우고 그 안에서 화면을 직접 갈아 끼웠다. Unity에서는 **여러 Scene을 갈아 끼우는** 구조라 화면별 코드 분리가 자연스럽다.

≡ Lifecycle / Event Functions

메서드	호출 시점
Awake()	오브젝트 생성 시 한 번 (비활성 상태에서도 호출)
OnEnable()	오브젝트가 활성화될 때마다
Start()	첫 Update 직전, 한 번
Update()	매 프레임 (보통 30~120fps, 불규칙)
FixedUpdate()	고정 시간 간격(0.02초)마다 --- 물리
LateUpdate()	모든 Update 후 --- 카메라 추적
OnDisable()	비활성화될 때
OnDestroy()	삭제 직전

생명 주기 흐름도



🔄 흐름 요약

초기화 3단계 → 매 프레임 FixedUpdate → Update → LateUpdate 반복 → 오브젝트가 비활성화/삭제되면 OnDisable → OnDestroy로 마무리.

Awake vs Start --- 둘 다 “초기화” 인데?

☰ Scene에 A, B, C 세 오브젝트가 있을 때 호출 순서

- 1 A.Awake → B.Awake → C.Awake
모든 오브젝트의 “기본 준비”가 끝남
- 2 A.Start → B.Start → C.Start
다른 오브젝트가 이미 Awake되어 있음이 보장됨
- 3 Update 루프 시작

💡 규칙

- Awake: 자기 자신의 변수 초기화, 같은 GameObject의 컴포넌트 캐싱
- Start: 다른 오브젝트와 상호작용해야 하는 초기화

Update vs FixedUpdate

두 메서드의 비교

	Update	FixedUpdate
호출 간격	프레임마다 (불규칙)	고정 (0.02초, 50회/초)
용도	입력, 일반 로직, 비물리 이동	Rigidbody2D 물리
시간 변수	<code>Time.deltaTime</code>	<code>Time.fixedDeltaTime</code>

✓ 어디에 무엇을 쓸까?

- 키보드 입력 검사 → Update
- `transform.position` 직접 변경하는 이동 → Update
- `Rigidbody2D.AddForce`, velocity 변경 → FixedUpdate
- 카메라가 캐릭터 따라가기 → LateUpdate
(캐릭터 Update가 끝난 후 따라가야 끊김 없음)

Rigidbody2D와 물리 엔진

⚙️ “이 오브젝트는 물리 법칙의 영향을 받는다”

Rigidbody2D가 붙은 오브젝트에 자동 적용: **중력**(자유 낙하), **관성**(마찰 없으면 계속 움직임), **충돌 반응**(Collider와 부딪히면 튕김).

↔ 일반 C#과 비교

일반 C#: “공이 벽에서 튕어 나오기” = `if (x < 0 || x > maxX) dx = -dx;`를 직접 작성.

Unity: Rigidbody2D + Collider2D만 붙이면 **한 줄도 안 쓰고** 튕긴다.

! 왜 FixedUpdate?

물리 엔진은 **고정 간격**으로 계산해야 정확. 프레임 속도(가변)에 맞추면 PC 성능에 따라 결과가 달라짐.

Time.deltaTime --- 프레임 보정

🕒 프레임 속도와 무관한 일정 속도

```
1 void Update()
2 {
3     // BAD: speed depends on frame rate
4     transform.position += new Vector3(0.1f, 0, 0);
5
6     // GOOD: 5 units per second, regardless of fps
7     transform.position += new Vector3(5f * Time.deltaTime, 0, 0);
8 }
```

📄 Time.deltaTime이란?

“이전 프레임으로부터 지금까지 경과한 시간(초)”.

- 60fps → $\text{Time.deltaTime} \approx 0.0167\text{s}$
- 30fps → $\text{Time.deltaTime} \approx 0.0333\text{s}$

이동량 = 속도 × 시간 이므로, 속도 * Time.deltaTime을 더하면 프레임 속도와 무관하게 같은 속도로 움직인다.

객체를 만드는 두 가지 방식

일반 C# (상속/직접 작성)

```
1 class Player {  
2     public float X, Y;  
3     void Update(...) { ... }  
4     void Draw(...) { ... }  
5 }
```

Unity (컴포넌트 조합)

```
1 GameObject "Player"  
2     + Transform      (position, rotation, scale)  
3     + SpriteRenderer (image)  
4     + Rigidbody2D    (physics)  
5     + BoxCollider2D  (collision shape)  
6     + PlayerController (our script)
```

상속(Inheritance) vs 조합(Composition)

🌲 상속 = “무엇이다”(is-a)

Player는 FlyingCharacter이다 → Movable이다 → ...

새 기능 추가 = 새 자식 클래스 → **상속 지옥(deep inheritance)**

조합이 어려운 다중 능력(“건고 날고 쏘는” 캐릭터) 표현이 까다롭다.

🧩 조합 = “무엇을 가진다”(has-a)

GameObject가 Mover, Shooter, HealthComponent를 가진다.

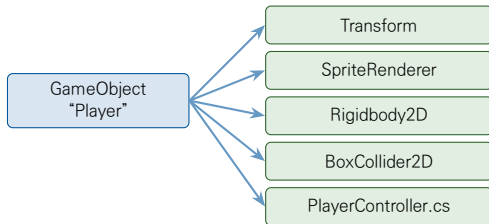
“걷는 적” 필요 → 적에 Mover만 붙이면 끝.

레고 블록처럼 기능을 조립.

★ Unity의 선택

Unity는 의도적으로 **조합**을 선택했다. 결과: 각 컴포넌트가 **한 가지 일만** 한다 → 재사용성/유연성/단순성이 함께 올라간다.

컴포넌트 모델 개념도



💡 핵심

GameObject는 빈 컨테이너일 뿐. "무엇을 할 수 있는가"는 거기에 붙어 있는 Component들의 합집합이 결정한다.

GetComponent --- 다른 컴포넌트 가져오기

 PlayerController.cs

```
1 public class PlayerController : MonoBehaviour
2 {
3     Rigidbody2D rb;    // cached reference
4     SpriteRenderer sr;
5
6     void Awake()
7     {
8         rb = GetComponent<Rigidbody2D>();
9         sr = GetComponent<SpriteRenderer>();
10    }
11
12    void Update()
13    {
14        if (Input.GetKeyDown(KeyCode.Space))
15        {
16            rb.velocity = new Vector2(0, 5);
17            sr.color = Color.red;
18        }
19    }
20 }
```

Update에서 GetComponent를 호출하지 마라

⚠ Bad vs Good

```
1 // BAD: searches every frame (60+ times per second)
2 void Update() {
3     GetComponent<Rigidbody2D>().velocity = ...;
4 }
5
6 // GOOD: search once, reuse the reference
7 Rigidbody2D rb;
8 void Awake() { rb = GetComponent<Rigidbody2D>(); }
9 void Update() { rb.velocity = ...; }
```

★ 기본 성능 규칙

찾는 것은 한 번, 사용하는 것은 매번

이 패턴은 Unity 스크립트의 거의 모든 곳에서 반복된다.

Vector3 --- 3D 좌표/방향

Vector 연산

```
1 Vector3 p = new Vector3(2f, 5f, 0f);
2
3 // Vector arithmetic
4 Vector3 a = new Vector3(1, 0, 0);
5 Vector3 b = new Vector3(0, 1, 0);
6 Vector3 c = a + b;      // (1, 1, 0)
7 Vector3 d = a * 5f;     // (5, 0, 0)
8
9 // Useful constants
10 Vector3.zero;           // (0, 0, 0)
11 Vector3.up;             // (0, 1, 0)
12 Vector3.right;          // (1, 0, 0)
```

i 왜 묶어서 다루나?

좌표를 x, y, z로 따로 두지 않고 Vector3로 묶어 다루면

(1) transform.position = new Vector3(...) 한 줄로 위치 설정,

(2) +, -, * 같은 벡터 연산을 자연스럽게 사용 가능.

2D 게임에서는 Vector2(x, y)를 주로 사용.

transform --- 모든 GameObject의 필수 컴포넌트

↔ Mover.cs

```
1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;
4
5     void Update()
6     {
7         // 3 units per second to the right
8         transform.position += new Vector3(speed * Time.deltaTime, 0, 0);
9
10        // 90 degrees per second around Z
11        transform.Rotate(0, 0, 90f * Time.deltaTime);
12    }
13 }
```

💡 transform, gameObject는 무료

MonoBehaviour가 이미 두 속성을 가지고 있다. GetComponent<Transform>()을 부를 필요 없이 그냥 transform이라고 쓰면 된다.

Inspector에 노출되는 필드

👁 노출 규칙

```
1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;        // shown
4     public Color color = Color.red; // shown
5     public GameObject target;       // shown (drag-drop)
6
7     private int hiddenValue = 10;   // NOT shown
8 }
```

🏠 직렬화 규칙

- public 필드 → 자동 노출
- private 필드 → 노출 안 됨
- [SerializeField] private → **노출됨**
- public + [HideInInspector] → 숨김
- 프로퍼티(public int X { get; set; }) → **노출 안 됨**

권장 패턴: [SerializeField] private

★ 한 줄로 두 마리 토끼

```
1 public class Player : MonoBehaviour
2 {
3     // Editable in Inspector, but NOT accessible from other scripts
4     [SerializeField] private float speed = 3f;
5     [SerializeField] private int maxHp = 100;
6
7     // Read-only access from outside
8     public int CurrentHp { get; private set; }
9 }
```

✓ 두 가지를 동시에 만족

- Inspector에서 조정 가능 (디자이너 편의 / 빠른 튜닝)
- 외부에서 접근 불가 (캡슐화 유지)

OOP의 캡슐화 원칙을 Unity 환경에 맞게 가져온 표준 패턴이다.

Inspector가 있다는 것의 의미

✎ 일반 C#과의 결정적 차이

일반 C# 프로그램에서 “플레이어 속도”를 바꾸려면

→ 코드 수정 → **재컴파일** → 실행 → 확인 → 다시 수정 ...

한 번 사이클에 수십 초 걸린다.

Unity에서는 [SerializeField] float speed만 선언해 두면

→ Inspector에서 슬라이더 드래그 → **즉시 반영**

플레이 중에도 값을 바꿔 가며 “느낌”을 잡을 수 있다.

✎ 디자이너와의 협업

프로그래머가 [SerializeField]로 “조정 가능한 손잡이”를 제공하면, 디자이너/기획자가 **코드를 모르고도** 게임의 느낌을 직접 다듬을 수 있다.

(1) MonoBehaviour는 new로 만들지 마라

❌ 잘못된 vs 올바른

```
1 // WRONG: compiles, but Unity ignores it
2 PlayerController p = new PlayerController();
3
4 // CORRECT: attach to a GameObject
5 GameObject obj = new GameObject("Player");
6 PlayerController p = obj.AddComponent<PlayerController>();
7
8 // ALSO CORRECT: instantiate from a Prefab
9 GameObject p2 = Instantiate(playerPrefab);
```

⚠ new로 만들면

- Unity가 그 존재를 모름 → Update가 호출되지 않음
- transform, gameObject가 null
- Destroy(this)도 동작하지 않음

AddComponent vs Instantiate vs Prefab

✂ 세 가지의 역할

- `AddComponent<T>()` --- 이미 존재하는 `GameObject`에 **컴포넌트 하나 추가**
- `Instantiate(prefab)` --- Prefab을 **복제**하여 Scene에 새 `GameObject` 생성
- Prefab(**프리팹**) --- 재사용 가능한 `GameObject` 템플릿(.prefab 파일)

💡 언제 무엇을 쓰나?

- 단순 일회성 → `new GameObject + AddComponent`
- 자주 만드는 복잡한 객체(총알, 적, 아이템) → **Prefab + Instantiate**

↔ 일반 C#과의 비교

일반 C#에서 “적”을 만들 때 `new Enemy(x, y)`로 코드에서 직접 생성했다면, Unity에서는 “Enemy 프리팹”을 **디자이너가 시각적으로 만들어 두고 Instantiate만 호출한다.**

(2) Update에서 무거운 작업을 하지 마라

⚠ 매 프레임 호출되므로 피해야 할 것들

- `GameObject.Find(...)` --- Scene 전체 검색
- `FindObjectOfType<T>()` --- 모든 오브젝트 순회
- `GetComponent<T>()` --- 컴포넌트 목록 검색
- `new SomeClass()` 반복 --- GC 부담
- `Debug.Log(...)` 남발 --- 의외로 비싸다

★ 원칙

찾는 것은 한 번, 사용하는 것은 매번

Awake/Start에서 검색 → 필드에 캐싱 → Update에서 재사용

가비지 컬렉션(GC)이 게임에서 왜 문제인가?

문제: 프레임 끊김

GC는 메모리 청소를 위해 **프로그램을 잠시 멈춘다**.

60fps → 한 프레임 16ms뿐인데 GC가 5ms만 발생해도 **끊김(stutter)**이 보인다.

원인: 반복적인 객체 생성

`new List<>()`, `new string()`을 매 프레임 만드는 것
(`Vector3`, `Vector2`는 구조체라 GC와 무관)

해결

- 필드로 미리 만들어 두고 재사용
- `string` + 대신 `StringBuilder`
- 자주 생성/파괴되는 것 → **Object Pooling**(객체 풀) 패턴

(3) Unity의 null은 일반 C#의 null과 다르다

가짜 null (Fake Null)

```
1  GameObject obj = ...;
2  Destroy(obj); // marked for destruction
3
4  // One frame later:
5  if (obj == null) // TRUE (Unity reports null)
6      Debug.Log("destroyed");
7
8  // But the C# reference is technically not null!
9  object asObj = obj;
10 if (asObj == null) // FALSE
11     ...
```

! 실용적 규칙

항상 == null로 비교하라.

?. (null 조건) 과 ?? (null 합치기)는 “가짜 null”을 인식하지 못해 파괴된 오브젝트의 메서드를 호출할 수 있다.

(4) Coroutine --- 시간이 흐르는 작업

Bomb.cs

```
1 using System.Collections;
2 using UnityEngine;
3
4 public class Bomb : MonoBehaviour
5 {
6     void Start() { StartCoroutine(ExplodeAfterDelay()); }
7
8     IEnumerator ExplodeAfterDelay()
9     {
10         Debug.Log("Bomb planted");
11         yield return new WaitForSeconds(2f);
12         Debug.Log("BOOM!");
13         Destroy(gameObject);
14     }
15 }
```

yield return 종류

- `yield return null;` --- 다음 프레임까지 대기
- `yield return new WaitForSeconds(2f);` --- 2초 대기
- `yield return new WaitUntil(() => isReady);` --- 조건이 참이 될 때까지

Coroutine 개념: 일반 C#과의 비교

❓ 일반 C#이라면 어떻게?

“2초 후 폭발”을 일반 C#으로 짜려면

- (a) `Thread.Sleep(2000)` → 프로그램이 멈춰서 입력도 못 받음
- (b) `Timer` 콜백 → 콜백 지옥, 상태 관리 복잡
- (c) `async/await` → 가능하지만 게임 루프와 잘 안 맞음

✓ Coroutine의 장점

- 게임 루프에 자연스럽게 녹아들 --- 각 `yield`가 “프레임 단위로 잘게 쪼개진 일”
- 상태 변수가 필요 없음 --- 메서드 안의 지역 변수가 자동 보존됨
- 쓰레드 아님 --- 메인 쓰레드에서 안전하게 실행, 동기화 걱정 없음

(5) 직접 그리지 않는다

Unity에서 그리기는 “컴포넌트의 일”

이미지 → `SpriteRenderer`, 3D 모델 → `MeshRenderer`, UI 텍스트 → `TextMeshProUGUI`,
선/궤적 → `LineRenderer`, 파티클 → `ParticleSystem`

스크립트의 역할

스크립트는 “어디에/어떻게” 그릴지만 결정 → `transform.position`, `sr.color`, `sr.sprite` 등을 변경. 실제 픽셀을 칠하는 것은 Unity의 렌더링 시스템.

일반 GUI와의 차이

일반 GUI: “매 프레임 Graphics 객체로 직접 그리기”를 호출. Unity: 어떤 그리기 코드도 작성하지 않는다.

Input.GetKey 세 가지 변형

세 메서드의 차이

메서드	언제 true
GetKey(K)	키 눌러 있는 동안 매 프레임 (이동)
GetKeyDown(K)	키 누른 그 프레임만 (점프, 발사)
GetKeyUp(K)	키 땀 그 프레임만 (차징 해제)

사용 예시

```
1 void Update() {  
2     if (Input.GetKey(KeyCode.LeftArrow))  
3         transform.Translate(-5 * Time.deltaTime, 0, 0);  
4     if (Input.GetKeyDown(KeyCode.Space)) Shoot();  
5 }
```

흔한 실수

점프에 GetKey를 쓰면 Space 누르는 동안 매 프레임 점프 → 미친 듯이 튼. “한 번 누름 = 한 번 동작”에는 GetKeyDown.

Collider와 충돌 콜백



충돌 콜백

```
1 void OnCollisionEnter2D(Collision2D c) {  
2     // physical collision (bounces)  
3     Debug.Log("Hit: " + c.gameObject.name);  
4 }  
5  
6 void OnTriggerEnter2D(Collider2D c) {  
7     // pass-through zone (no bounce)  
8     Debug.Log("Entered: " + c.gameObject.name);  
9 }
```



Collision vs Trigger

- **Collision** --- 실제로 부딪히고 튕김 (벽, 바닥)
- **Trigger** --- 통과는 되지만 알림은 받음 (아이템 영역)
- Collider의 **Is Trigger** 체크박스로 모드 전환

Unity가 대신 해 주는 것들

★ “코드가 짧다”고 느끼는 이유

Unity는 다음을 **자동으로** 처리한다:

- 진입점, 메시지 루프, 창 생성
- 프레임 타이머, 더블 버퍼링, 화면 갱신
- 키보드/마우스/터치 입력의 저수준 처리
- 물리 시뮬레이션 (중력, 충돌, 마찰)
- 텍스처/메시 로딩, 셰이더, 렌더링 파이프라인
- 오디오, 네트워크, 파일 시스템 추상화

✓ 우리가 작성하는 부분

오직 **내 게임의 규칙**만.

이것이 게임 엔진의 가치이고, Unity 스크립트가 짧고 “조각” 처럼 보이는 이유이다.

한눈에 보는 비교: 일반 C# vs Unity

田 같은 일을 어떻게 하나

	일반 C#/GUI 프로그램	Unity
진입점	Main()	없음 (엔진이 자동)
게임 루프	while/Timer 직접 작성	Update 자동
프레임 보정	직접 시간 측정	Time.deltaTime
화면 갱신	Invalidate/Repaint	자동 렌더링
그리기	Graphics.FillRectangle 등	SpriteRenderer
객체 모델	클래스 직접 작성	GameObject + Component
입력 처리	이벤트 핸들러	Input.GetKey 폴링
충돌 판정	Math.Abs(...) < N 직접	Collider + 콜백
값 조정	코드 수정 후 재컴파일	Inspector에서 즉시

● 동작 명세

화면에 여러 공이 떠다니다가:

- 벽에 부딪히면 **튕김** (자동, 우리 코드 없음)
- 공끼리 부딪히면 **하나로 합쳐짐** (우리 코드)

⚙ 물리 규칙 (보존 법칙)

- 면적 보존: $r_{\text{new}} = \sqrt{r_1^2 + r_2^2}$
- 질량 보존: $m_{\text{new}} = m_1 + m_2$
- 운동량 보존: $\vec{v}_{\text{new}} = \frac{m_1 \vec{v}_1 + m_2 \vec{v}_2}{m_1 + m_2}$
- 위치 = **질량 중심**

이 예제로 익히는 개념

✔ 한 번에 적용되는 개념

- MonoBehaviour, Awake/Start 초기화 패턴
- Rigidbody2D + Collider2D로 자동 물리/충돌
- OnCollisionEnter2D 충돌 콜백
- Prefab + Instantiate로 다수 객체 생성
- [SerializeField]로 Inspector 노출
- Destroy로 오브젝트 제거
- GetComponent 캐싱

Hierarchy 설계

Scene Hierarchy

```
1 Main Camera          // Unity default
2 GameManager          // empty GameObject
3   + BallSpawner.cs   // our script
4 Walls                // empty group
5   + WallTop           // BoxCollider2D
6   + WallBottom
7   + WallLeft
8   + WallRight
```

설계 포인트

- 공(Ball)은 Hierarchy에 두지 않음 → **Prefab**으로 만들고 BallSpawner가 Instantiate
- 빈 GameObject Walls로 4개 벽을 묶어 정리 (SetActive, 이동을 한꺼번에)

BallPrefab 설계

공에 필요한 컴포넌트

컴포넌트	역할
Transform	위치/크기 (자동)
SpriteRenderer	원 모양 표시
CircleCollider2D	원형 충돌 영역
Rigidbody2D	물리 (관성, 충돌 응답)
Ball.cs	우리 스크립트

Rigidbody2D 설정

Gravity Scale = 0 (떨어지지 않게), Linear Drag = 0 (계속 움직이게), Collision Detection = Continuous (빠른 공 통과 방지)

Physics Material 2D Bouncy

Friction = 0, Bounciness = 1 (완전 탄성 충돌)

스크립트 설계 --- 역할 분리

두 개의 스크립트

Ball.cs --- 개별 공의 동작

- 시작 시 무작위 방향 발사
- 다른 공과 충돌하면 합치기

BallSpawner.cs --- 전체 게임 관리

- 시작 시 N개의 공 생성

충돌 시 “누가 합치는가”?

A↔B 충돌 시 OnCollisionEnter2D가 **양쪽에서** 호출됨.

→ GetInstanceID()가 **더 작은 쪽만** 처리하도록 분기.

Ball.cs --- 초기화와 충돌 감지

Ball.cs (1/2)

```
1 public class Ball : MonoBehaviour
2 {
3     [SerializeField] private float initialSpeed = 3f;
4     private Rigidbody2D rb;
5
6     void Awake() { rb = GetComponent<Rigidbody2D>(); }
7
8     void Start()
9     {
10         float angle = Random.Range(0f, 360f) * Mathf.Deg2Rad;
11         Vector2 dir = new Vector2(Mathf.Cos(angle), Mathf.Sin(angle));
12         rb.velocity = dir * initialSpeed;
13     }
14
15     void OnCollisionEnter2D(Collision2D c)
16     {
17         Ball other = c.gameObject.GetComponent<Ball>();
18         if (other == null) return; // hit a wall
19         if (GetInstanceID() > other.GetInstanceID()) return;
20         Merge(other);
21     }
22     // ... Merge() on next slide
23 }
```

Ball.cs --- Merge 메서드

Ball.cs (2/2)

```
1 void Merge(Ball other)
2 {
3     float r1 = transform.localScale.x * 0.5f;
4     float r2 = other.transform.localScale.x * 0.5f;
5     float rNew = Mathf.Sqrt(r1*r1 + r2*r2); // area conserved
6
7     float m1 = rb.mass, m2 = other.rb.mass;
8     float mNew = m1 + m2; // mass conserved
9
10    Vector2 vNew = (m1*rb.velocity + m2*other.rb.velocity) / mNew;
11    Vector2 pNew = (m1*(Vector2)transform.position
12                  + m2*(Vector2)other.transform.position) / mNew;
13
14    transform.position = pNew;
15    transform.localScale = Vector3.one * (rNew * 2f);
16    rb.mass = mNew;
17    rb.velocity = vNew;
18
19    Destroy(other.gameObject); // remove the other
20 }
```

BallSpawner.cs



BallSpawner.cs

```
1 public class BallSpawner : MonoBehaviour
2 {
3     [SerializeField] private GameObject ballPrefab;
4     [SerializeField] private int count = 10;
5     [SerializeField] private Vector2 areaMin = new Vector2(-7,-4);
6     [SerializeField] private Vector2 areaMax = new Vector2( 7, 4);
7
8     void Start()
9     {
10         for (int i = 0; i < count; i++)
11         {
12             float x = Random.Range(areaMin.x, areaMax.x);
13             float y = Random.Range(areaMin.y, areaMax.y);
14             Instantiate(ballPrefab,
15                       new Vector3(x, y, 0),
16                       Quaternion.identity);
17         }
18     }
19 }
```

Editor 셋업: Step-by-Step (1/2)

⚙ Step 1--3

Step 1: 새 2D 프로젝트

Unity Hub → New Project → "2D Core"

Step 2: 벽 만들기

- Walls 빈 GameObject 생성
- 그 아래 2D Object > Sprites > Square 4개 (Top/Bottom/Left/Right)
- Scale/Position 조정 + BoxCollider2D 추가

Step 3: BallPrefab

- 2D Object > Sprites > Circle 생성 → Ball
- Rigidbody2D (Gravity 0) + CircleCollider2D 추가
- Bouncy Physics Material 2D 만들어 Collider에 할당
- Ball.cs 부착 → Project로 드래그하여 Prefab화

⚙ Step 4--5

Step 4: GameManager + Spawner

- 빈 GameObject GameManager 생성
- BallSpawner.cs 부착
- Inspector의 Ball Prefab 슬롯에 BallPrefab 드래그

Step 5: Play

- 상단 ▶ Play 버튼 클릭
- 10개의 공이 튕기다가 부딪히면 큰 공 하나로 합쳐지는 것 확인

💡 작성한 코드의 양

실제 우리가 작성한 코드는 약 60줄.
같은 게임을 일반 C#/GUI로 짤다면 수백 줄 필요.

무엇이 어디서 일어나는가

田 책임 분담표

이벤트	누가 처리하나
공 생성	BallSpawner.Start
초기 속도 설정	Ball.Start
매 프레임 위치 갱신	Rigidbody2D (자동)
벽 충돌 반사	Collider2D + Material (자동)
공-공 충돌 감지	OnCollisionEnter2D 콜백
합치기 계산	Ball.Merge
사라진 공 제거	Destroy(...)

★ 우리가 정말 작성한 부분

무작위 초기 속도 + 합치기 로직 --- 그게 전부.

나머지는 모두 Unity 컴포넌트가 처리한다.

단계별 진행

Phase	내용
1. 예제 변형	색/사운드 등 시각·청각 피드백 추가
2. 새 메커니즘	코루틴, UI, 입력으로 규칙 변경
3. 새 게임	Rigidbody/Collider/Prefab 종합 활용
4. 자유 창작	5개 이상 개념을 활용해 완성

제출

프로젝트 압축(Library/, Temp/, Logs/, obj/ 제외)

+ README.md + 화면 캡처 1~2장

Phase 1 --- 예제 변형 (기초)

코드를 조금만 고쳐 결과를 크게

❶ 색상 평균 합치기

```
sr.color = (sr.color + other.sr.color) * 0.5f;
```

❷ 속도에 따른 색 변화

```
Color.Lerp(Color.blue, Color.red, |v| / maxSpeed)
```

❸ 충돌 사운드

```
AudioSource.PlayOneShot(wallHit / mergeHit)
```

벽인지 공인지 GetComponent<Ball>() == null로 분기

Phase 1의 목표

Inspector에서 즉시 반영되는 감각, “코드 한 줄로 결과가 달라짐”을 체득하는 단계.

Phase 2 --- 새 메커니즘 추가 (응용)

새 컴포넌트와 코루틴 도입

1 마우스 클릭으로 공 추가

```
1 if (Input.GetMouseButtonDown(0)) {  
2     Vector3 mp = Input.mousePosition;  
3     mp.z = -Camera.main.transform.position.z;  
4     Vector3 w = Camera.main.ScreenToWorldPoint(mp);  
5     w.z = 0;  
6     Instantiate(ballPrefab, w, Quaternion.identity);  
7 }
```

2 최대 크기 분열 --- 반지름 > maxRadius이면 둘로 쪼개기 (면적/운동량 보존)

3 코루틴 자동 스폰 --- IEnumerator + yield return new WaitForSeconds(2f)

4 HUD 점수 (싱글톤 + TextMeshProUGUI)

```
GameManager.Instance.AddScore(1)
```

Phase 3 --- 새 게임 (종합 응용)

📁 과제 3.1 — 벽돌 깨기 (Block Breaker)

“공 합치기” 자산(BallPrefab, Bouncy)을 **재사용**.

- Paddle.cs --- 좌우 키 이동
- Brick.cs --- 충돌 시 Destroy(gameObject), 점수 +10
- Ball.cs --- 화면 아래로 떨어지면 Game Over
- 모든 벽돌 사라지면 Stage Clear

🔧 과제 3.2 — 중력 토글 시뮬레이터

SPACE 키로 Physics2D.gravity = -Physics2D.gravity; 토글.

합치기 로직은 그대로 유지. 토글 시 모든 공이 한 번 반짝이도록 코루틴으로 시각화.

📄 필수: README에 비교 글

같은 기능을 일반 C#/GUI로 짤 때의 **코드 줄 수**와 본 과제에서 **우리가 작성한 줄 수**를 비교. 사라진 코드가 어떤 **컴포넌트**로 옮겨갔는지 짚지어 설명.

Phase 4 --- 자유 창작

💡 5개 이상 개념을 활용한 미니 게임

필수 개념 (5개 이상)

[SerializeField], GetComponent 캐싱, Rigidbody2D, Prefab + Instantiate, OnCollisionEnter2D, Coroutine, Time.deltaTime, TextMeshProUGUI

🎮 주제 예시

- Catch the Star --- 떨어지는 별 받기
- Mini Shooter --- 적이 내려오고 플레이어가 발사
- Dodge --- 떨어지는 공 피하기
- Top-Down Maze --- 미로 탈출
- Flappy 스타일 --- 클릭으로 점프, 장애물 회피

★ 팁

한 가지 메커니즘만 **확실히** 동작시킨 뒤 확장하라. Phase 1~3 자산 재사용 권장.

✔ 이번 강의에서 배운 것

개념

MonoBehaviour

생명 주기

컴포넌트 모델

GetComponent 캐싱

Time.deltaTime

Inspector 직렬화

new 금지

가짜 null

Coroutine

Unity가 인식하는 스크립트가 되기 위한
Awake/Start/Update/FixedUpdate/LateUpdate

상속이 아닌 **조합**으로 GameObject

Update에서 매번 호출

프레임 속도와 무관한

public / [SerializeField]

GetComponent 또는 Instantiate

항상 == null로 안전

시간 흐름이 있는

≡ 직접 해 보기

- 1 Unity Editor에서 LifecycleDemo 스크립트를 붙이고 Console에서 Awake → Start 순서를 확인
- 2 Mover 스크립트를 작성하여 speed를 Inspector에서 조정
- 3 “화살표 키로 움직이는 캐릭터”를 작성
(Player.cs에서 키 입력 + transform.position 변경)
- 4 Bomb 코루틴을 변형하여 1초 간격으로 Debug.Log를 5번 출력 후 오브젝트가 사라지게 하라

Part II

Unity로 체험하는 C#의 핵심 기능들

C# Feature Lab --- 작은 실험들의 모음

Unity는 “무대” 다

이 과목의 주인공은 C#.

Unity는 `Console.WriteLine`보다 훨씬 친절한 무대일 뿐.

💡 콘솔 vs Unity

- 콘솔: `Console.WriteLine("HP: " + hp)` → 글자만
- Unity: `cube.color = Color.red` → 큐브가 빨개진다

같은 `int`, 같은 `enum`, 같은 `event`라도
눈으로 보면 머리에 더 잘 새겨진다.

이번 강의의 형식

큰 게임 한 편을 만들지 않는다.
대신 6개의 작은 실험(Lab)을 만든다.

각 Lab의 형식:

- ❶ 문제: “이런 상황이라면 어떻게?”
- ❷ 콘솔 비교: 일반 C# 콘솔이라면
- ❸ Unity 데모: 짧은 스크립트 + 키 입력
- ❹ 실험해 보기: 한 줄 바꾸고 결과 관찰

게임을 “완성”하는 것이 목표가 아니라
C# 기능 하나하나를 손에 익히기가 목표.

Lab	C# 기능
1	enum + switch 식
2	프로퍼티, 캡슐화
3	interface
4	event / Action<T>
5	제네릭 List<T> + LINQ
6	Coroutine + 람다

Unity Hub → New Project → 3D (또는 2D) → CSharpFeatureLab

씬에 미리 만들어 둘 GameObject:

- Main Camera, Directional Light (기본값)
- LabCube --- 3D Cube, (0,0,0) (Lab 1, 4에서 사용)
- LabRoot --- 빈 GameObject, Lab 스크립트들을 붙임
- Canvas + LogText (선택) --- 화면 위에 결과 표시

콘솔 창(Window → General → Console)을 열어 두고
Debug.Log와 **화면 변화를 동시에** 본다.

세 가지 “기분(mood)” 중 하나를 표현하고 싶다.

int 0/1/2로 쓰면...

- “state == 10이 무슨 뜻이지?”
- 오타도 잡히지 않음

→ 이름 있는 상수 집합이 필요

→ enum

Lab1_Enum.cs

```
1 public enum Mood { Happy, Sad, Angry }
2
3 public class Lab1_Enum : MonoBehaviour
4 {
5     [SerializeField] private Renderer targetRenderer;
6
7     void Update()
8     {
9         if (Input.GetKeyDown(KeyCode.Alpha1)) Apply(Mood.Happy);
10        if (Input.GetKeyDown(KeyCode.Alpha2)) Apply(Mood.Sad);
11        if (Input.GetKeyDown(KeyCode.Alpha3)) Apply(Mood.Angry);
12    }
13
14    void Apply(Mood m)
15    {
16        Color color = m switch
17        {
18            Mood.Happy => Color.yellow,
19            Mood.Sad   => new Color(0.3f, 0.5f, 1f),
20            Mood.Angry => Color.red,
21            -           => Color.white
22        };
23        targetRenderer.material.color = color;
24        Debug.Log($"Mood = {m}");
25    }
26 }
```


switch 표현식 (C# 8+)

```
1 // Old style (statement)
2 Color color;
3 switch (m) {
4     case Mood.Happy: color = Color.yellow; break;
5     case Mood.Sad:   color = Color.blue;   break;
6     default:        color = Color.white; break;
7 }
8
9 // New style (expression)
10 Color color = m switch {
11     Mood.Happy => Color.yellow,
12     Mood.Sad   => Color.blue,
13     -         => Color.white
14 };
```

값을 반환하는 switch.

“무엇을 얻는가”가 한눈에.

Lab 1 Unity 구현 가이드

- 1 Hierarchy → 3D Object → Cube → 이름 MoodCube, 위치 (0,0,0)
- 2 Hierarchy → Create Empty → 이름 Lab1
- 3 Assets/Scripts/Lab1_Enum.cs 작성
- 4 스크립트를 Lab1 객체에 **드래그**해 부착
- 5 Inspector의 Target Renderer 슬롯에 MoodCube를 **드래그**
- 6 Play → 키 1, 2, 3

? 왜 GameObject 드래그가 Renderer로?

Unity가 자동으로 **그 GameObject의 Renderer 컴포넌트**를 찾아 채움.

```
= moodCube.GetComponent<Renderer>()
```

기초 문법: [SerializeField] private

- private 필드 = 같은 클래스 안에서만 접근
- Inspector에서도 안 보임 → 디자이너 친화적이지 않음
- [SerializeField] = “Inspector에 보여 주세요” 어트리뷰트

SerializeField 효과

[SerializeField] private float speed = 8f;

- 다른 클래스에서 lab.speed 접근 X
- Inspector에서 값 수정 O

[Foo] 형태는 모두 어트리뷰트

--- 클래스/필드/메서드에 “꼬리표” 붙이기

기초 문법: 문자열 보간 "\$..."

```
1 int n = 42;
2 string s = $"n = {n}";      // "n = 42"
3 string t = $"sum = {a + b}"; // expressions also work
4 string u = $"avg = {x:F2}";  // F2 = two decimal places
5 string v = $"{{score:D6}}";  // D6 = at least 6 digits, pad 0
```

- \$ 접두어 + {} 안에 식
- 콜론 뒤는 형식 지정자 (F2, D6 등)
- 옛 string.Format, "x = " + n보다 가독성 우수

실험해 보기 (Lab 1)

- `Mood.Excited`를 추가하고 키 4에 연결.
switch의 `_` 가지가 어떤 역할인지 확인
- `(int)Mood.Happy`는 무엇? `Debug.Log`로 출력
- `enum Mood : byte`로 바꾸면 무엇이 달라질까?
(저장 크기)

문제: 누구나 HP를 바꾸면?

```
1 class Character
2 {
3     public int hp; // BAD: anyone can write
4 }
5
6 var c = new Character();
7 c.hp = 100;
8 c.hp = -50; // negative is OK?
9 c.hp = 999999; // god mode
```

외부 읽기 OK, 외부 쓰기 차단이 필요.

→ `public int Hp { get; private set; }`

Lab2_Property.cs

```
1 public class Lab2_Property : MonoBehaviour
2 {
3     [SerializeField] private int maxHp = 10;
4     public int Hp { get; private set; } // <<< the key
5
6     void Start() { Hp = maxHp; }
7
8     void Update()
9     {
10         if (Input.GetKeyDown(KeyCode.D)) { Damage(1); Report(); }
11         if (Input.GetKeyDown(KeyCode.H)) { Heal(1); Report(); }
12
13         // The line below would NOT compile, by design:
14         // if (Input.GetKeyDown(KeyCode.R)) Hp = 9999;
15     }
16
17     public void Damage(int amount) => Hp = Mathf.Max(0, Hp - amount);
18     public void Heal(int amount) => Hp = Mathf.Min(maxHp, Hp + amount);
19
20     void Report() => Debug.Log($"HP = {Hp}/{maxHp}");
21 }
```

프로퍼티 한 줄 옵션들

☰ 프로퍼티 접근 수준

- { get; set; } --- 읽기 OK, 쓰기 OK (필드와 비슷)
- { get; private set; } --- 외부 읽기 OK, **쓰기 차단**
- { get; } --- **읽기 전용** (생성자에서만 초기화)

=>로 한 줄 메서드:

```
1 public void Damage(int n) => Hp = Mathf.Max(0, Hp - n);  
2
```


Lab 2 Unity 구현 가이드

- 1 Hierarchy → Create Empty → 이름 Lab2
- 2 Assets/Scripts/Lab2_Property.cs 작성, Lab2에 부착
- 3 Inspector에 **Max Hp = 10**이 보임
([SerializeField] 덕분)
Hp는 **보이지 않음** ([SerializeField]가 없는 자동 프로퍼티)
- 4 Window → General → Console
- 5 Play → **D**로 데미지, **H**로 회복

Console에 HP = 9/10, HP = 8/10...

Hp가 **0 미만/maxHp 초과로 가지 않음** (Mathf.Max/Min)

기초 문법: { get; private set; }을 풀어 보면

```
1 // short form (auto-property)
2 public int Hp { get; private set; }
3
4 // expanded by the compiler
5 private int _hp;           // hidden backing field
6 public int Hp
7 {
8     get { return _hp; }     // public read
9     private set { _hp = value; } // private write
10 }
```

- value는 set이 받는 값을 가리키는 **약속된 키워드**
- Hp = 10; → set의 value에 10

기초 문법: =>의 세 얼굴 (1/2)

C#의 =>는 문맥에 따라 세 가지로 쓰인다.
모두 “왼쪽이 들어가서 오른쪽이 나온다”.

(1) 식 본문 멤버 (Lab 2, Lab 3에서)

“이 메서드/프로퍼티는 이 식 한 줄을 반환”

```
1 public void Damage(int n) => Hp -= n;           // method
2 public string Name => "Circle";                 // property
3 public float Area() => Mathf.PI * r * r;         // method
```

(2) 람다 식 (Lab 5, Lab 6에서)

“매개변수를 받아 식을 반환하는 함수”

```
1 x => x >= 80                                     // Func<int, bool>
2 n => Debug.Log($"Tick {n}")                      // Action<int>
3 () => 42                                          // Func<int>
```

기초 문법: =>의 세 얼굴 (2/2)

(3) switch 식의 가지 (Lab 1에서)

“이 패턴이면 이 값”

```
1 Color c = m switch {  
2     Mood.Happy => Color.yellow,  
3     Mood.Sad   => Color.blue,  
4     -          => Color.white  
5 };
```

 기억하는 법: 어디 있는지를 보라

- 메서드/프로퍼티 선언 자리 → (1) 식 본문 멤버
- 인자/변수 자리에서 함수 자체를 만든다 → (2) 람다
- switch 안 → (3) 가지(arm)

실험해 보기 (Lab 2)

- `public int Hp { get; private set; }` → `public int Hp;`로 바꾸면?
다른 스크립트에서 `lab.Hp = 9999` 가능해진다
- `Hp { get; }`로 바꾸면?
Damage 안의 `Hp = ...`도 컴파일 에러
→ “어디까지 쓰기 허용”을 한 줄로 조절하는 감각

문제: 도형마다 면적 공식이 다르다

$$\text{Circle.Area()} = \pi r^2$$

$$\text{Rectangle.Area()} = w \cdot h$$

$$\text{Triangle.Area()} = \frac{1}{2}bh$$

공식은 다르지만, “**면적을 구할 수 있다**”라는 **능력은 공통**.

→ “공통 능력”만 약속해 두면

도형이 늘어나도 호출 코드는 그대로.

IShape와 두 구현체

```
1 public interface IShape
2 {
3     string Name { get; }
4     float Area();
5 }
6
7 public class CircleShape : IShape
8 {
9     private float radius;
10    public CircleShape(float r) { radius = r; }
11    public string Name => "Circle";
12    public float Area() => Mathf.PI * radius * radius;
13 }
14
15 public class RectangleShape : IShape
16 {
17     private float w, h;
18    public RectangleShape(float w, float h) { this.w = w; this.h = h; }
19    public string Name => "Rectangle";
20    public float Area() => w * h;
21 }
```

```
1 public class Lab3_Interface : MonoBehaviour
2 {
3     private IShape[] shapes;
4
5     void Start()
6     {
7         shapes = new IShape[] {
8             new CircleShape(2f),
9             new RectangleShape(3f, 4f),
10            new CircleShape(1.5f),
11        };
12    }
13
14    void Update()
15    {
16        if (Input.GetKeyDown(KeyCode.A))
17        {
18            foreach (var s in shapes)
19                Debug.Log($"{s.Name}: area = {s.Area():F2}");
20        }
21    }
22 }
```


interface 핵심

“무엇을 할 수 있는가”만 약속.

어떻게는 구현 클래스 자유.

상속 vs 인터페이스

- 상속: “Cat **is** Animal” (is-a)
- 인터페이스: “Cat **can** 울다” (can-do)

“면적을 **구할 수 있다**” → can-do → 인터페이스

Lab 3 Unity 구현 가이드

- 1 Hierarchy → Create Empty → 이름 Lab3
- 2 Assets/Scripts/에 4개 파일 생성:
IShape.cs, CircleShape.cs,
RectangleShape.cs, Lab3_Interface.cs
- 3 Lab3_Interface.cs만 Lab3에 드래그
(다른 셋은 MonoBehaviour가 아니므로 부착 불필요)
- 4 Window → Console, Play → A

i 파일명 = 클래스명 규칙

MonoBehaviour만 적용.

인터페이스/일반 클래스는 한 파일에 모아도 OK.

하지만 “한 파일 한 클래스”가 읽기 좋다.

기초 문법: public string Name => "Circle";

식 본문 프로퍼티(expression-bodied property)

“이 프로퍼티는 항상 이 식의 값을 반환”.

```
1 // Short form
2 public string Name => "Circle";
3
4 // Same meaning, expanded
5 public string Name
6 {
7     get { return "Circle"; }
8 }
```

- set이 없으니 외부 쓰기 불가
- 메서드 아니라 프로퍼티 → 괄호 없이 circle.Name
- =>의 세 얼굴 중 (1) 식 본문 멤버

기초 문법: foreach와 var

```
1 foreach (var s in shapes)
2     Debug.Log($"{s.Name}: {s.Area():F2}");
```

- `foreach (T x in xs)`: `xs`의 원소를 하나씩 `x`로
- `var s = “타입을 컴파일러가 알아서”`
`shapes`가 `IShape[]`이니 `s`는 자동으로 `IShape`

⚠ 주의

`var`는 “아무 타입이나”가 아니다.
타입은 컴파일 시점에 결정. JS의 `var`와 다름.

실험해 보기 (Lab 3)

- TriangleShape를 추가, shapes에 넣는다.
→ Lab3_Interface는 **한 줄도 수정 X**
- IShape에 Perimeter()를 추가하면?
모든 구현체가 **컴파일 에러**.
약속 위반은 컴파일러가 막아 준다.

문제: 변수가 바뀌면 “여러 곳”에 알려야

숫자 n 이 바뀔 때:

- UI 텍스트 갱신
- 사운드 재생
- 색상 변경
- ...

변수 쪽에서 **직접 호출**하면
알림 받는 곳이 늘 때마다 코드가 **비대해진다**.

- “소문”만 내고 듣는 사람은 알아서
- event

Lab4_Counter.cs (발신자)

```
1 using System;
2 using UnityEngine;
3
4 public class Lab4_Counter : MonoBehaviour
5 {
6     public event Action<int> OnChanged;
7     private int value_ = 0;
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.Space))
12        {
13            value_++;
14            OnChanged?.Invoke(value_);
15        }
16    }
17 }
```

발신자는 누가 듣는지 모른다.

?.Invoke: 구독자 없을 때 안전 호출.

Lab4_Logger / ColorChanger (구독자)

```
1 public class Lab4_Logger : MonoBehaviour
2 {
3     [SerializeField] private Lab4_Counter source;
4
5     void OnEnable() { source.OnChanged += React; }
6     void OnDisable() { source.OnChanged -= React; }
7
8     void React(int v) => Debug.Log($"Logger heard: {v}");
9 }
10
11 public class Lab4_ColorChanger : MonoBehaviour
12 {
13     [SerializeField] private Lab4_Counter source;
14     [SerializeField] private Renderer target;
15
16     void OnEnable() { source.OnChanged += React; }
17     void OnDisable() { source.OnChanged -= React; }
18
19     void React(int v)
20     {
21         float t = (v % 10) / 10f;
22         target.material.color = Color.Lerp(Color.black, Color.cyan, t);
23     }
24 }
```


delegate, Action, event 한 묶음

- delegate: 메서드를 변수처럼 담는 그릇
- Action<T>: “T 받고 void” 미리 만들어 둔 delegate
- Func<T, R>: “T 받고 R 반환”
- event: 외부에서 +=/-=만 가능 (직접 호출 X)
- ?.Invoke(...): null 안전 호출

⚠ 필수 짝

OnEnable에서 +=, OnDisable에서 -=.
안 그러면 메모리 누수 + 파괴된 객체에 호출.

Lab 4 Unity 구현 가이드 (1/2)

이번 Lab은 여러 컴포넌트가 서로 참조 → 단계가 길다.

Hierarchy 구조:

```
Lab4_Source      <-- Lab4_Counter (발신자)
Lab4_Subscribers <-- Lab4_Logger
                  Lab4_ColorChanger
Lab4_Cube         <-- 3D Cube (색이 바뀜)
```

- 1 Lab4_Source (Empty) + Lab4_Counter.cs
- 2 Lab4_Cube (3D Cube), 위치 (2, 0, 0)
- 3 Lab4_Subscribers (Empty)
+ Lab4_Logger.cs, Lab4_ColorChanger.cs
(한 객체에 컴포넌트 두 개)

Lab 4 Unity 구현 가이드 (2/2)

Inspector 슬롯 연결 (가장 중요):

- `Lab4_Logger.Source` ← `Lab4_Source` 드래그
(Unity가 `Lab4_Counter` 컴포넌트를 자동 추출)
- `Lab4_ColorChanger.Source` ← `Lab4_Source`
- `Lab4_ColorChanger.Target` ← `Lab4_Cube`

Play → **SPACE**

콘솔에 **로그 + 큐브 색이 동시에 변함.**

두 구독자는 서로의 존재를 **모른다.**

기초 문법: Action<T>, Func<T, R>

```
1 // Pre-defined delegates in .NET
2 public delegate void Action<T>(T arg);
3 public delegate R Func<T, R>(T arg);
```

- Action --- 매개변수 없음, void 반환
- Action<int> --- int 받고 void
- Action<int, string> --- int+string 받고 void
- Func<int, bool> --- int 받고 bool 반환
- Func<int, int, int> --- 마지막이 반환 타입

매번 `public delegate void MyHandler(int);`
을 새로 정의할 필요가 없게 해 주는 단축.

기초 문법: ?.와 ?.Invoke(...)

```
1 string s = null;
2 int len = s.Length;    // NullReferenceException!
3 int? len2 = s?.Length; // if s is null, len2 is null. Safe.
4
5 // Equivalent two lines
6 OnChanged?.Invoke(value_);
7 if (OnChanged != null) OnChanged(value_);
```

- ?. --- “왼쪽이 null이 아닐 때만 오른쪽”
- event는 외부에서 직접 호출 불가 → Invoke()
- ?.Invoke(...) = 사실상 표준 패턴

기초 문법: OnEnable/OnDisable 시점

Lecture 9의 MonoBehaviour 생명 주기 중 두 개:

- **OnEnable** --- 컴포넌트 **활성화될 때**
Play 시작, 체크박스 켜기, `SetActive(true)`
- **OnDisable** --- 컴포넌트 **비활성화될 때**
체크박스 끄기, `SetActive(false)`, 객체 파괴 직전

Start/OnDestroy vs OnEnable/OnDisable

Start/OnDestroy로 짤 맞춰도 동작은 함.
하지만 **컴포넌트가 켜졌다 꺼졌다** 할 때
OnEnable/OnDisable이 **자동으로 호출**되어 더 안전.

문제: 점수들의 통계를 빠르게

여러 점수를 모아 두고...

- Count, Sum, Average, Max, Min
- “80점 이상이 몇 명?”

배열은 크기 고정 → List<T>

for 루프 직접? → LINQ 한 줄로

같은 List 클래스가 int, string, Mood...

무엇이든 담을 수 있다 → 제네릭

Lab5_Generic.cs

```
1 using System.Collections.Generic;
2 using System.Linq;
3 using UnityEngine;
4
5 public class Lab5_Generic : MonoBehaviour
6 {
7     private List<int> scores = new List<int>();
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.N))
12        {
13            int s = Random.Range(0, 100);
14            scores.Add(s);
15            Debug.Log($"Added: {s}, Count={scores.Count}");
16        }
17        if (Input.GetKeyDown(KeyCode.L))
18        {
19            if (scores.Count == 0) { Debug.Log("No scores yet."); return; }
20            int sum = scores.Sum();
21            double avg = scores.Average();
22            int top = scores.Where(x => x >= 80).Count();
23            Debug.Log($"Sum={sum}, Avg={avg:F1}, >=80: {top}");
24        }
25    }
26 }
```


제네릭 <T>: 타입을 변수처럼

- `List<int>`, `List<string>`, `List<Mood>`...
- `List<int>.Add("hi")`는 **컴파일 에러**
(옛 `ArrayList`는 런타임에야 발견)

LINQ: `using System.Linq;`로 마법 메서드들

- `.Sum()` `.Average()` `.Max()` `.Count()`
- `.Where(predicate)` --- 거르기
- `.Select(transform)` --- 변환
- `.OrderBy(key)` --- 정렬

`x => x >= 80`: **람다** (`Func<int,bool>`)

Lab 5 Unity 구현 가이드

❶ Hierarchy → Create Empty → 이름 Lab5

❷ Lab5_Generic.cs 작성 후 Lab5에 부착

❸ 파일 맨 위 **using** 두 줄 필수:

```
using System.Collections.Generic; // List<T>
using System.Linq;                // .Sum(), .Where()
```

❹ Console 열기, Play → **N/L/K**

- **N** --- 점수 추가 (0~99 랜덤)
- **L** --- 통계 (Sum/Avg/≥80)
- **K** --- 비우기

기초 문법: 람다 $x \Rightarrow x \geq 80$

Where는 `Func<int, bool>`을 인자로 요구.

```
1 IEnumerable<int> Where(Func<int, bool> predicate);
2
3 // Old way: define a method, pass as method group
4 bool IsHigh(int x) { return x >= 80; }
5 scores.Where(IsHigh);
6
7 // New way: lambda directly
8 scores.Where(x => x >= 80);
```

컴파일러가 x 는 `int`, 반환은 `bool`로 자동 추론.

람다의 모양들:

```
1 x => x + 1           // single param: parentheses optional
2 () => 42              // no params
3 (a, b) => a + b       // multiple params
4 x => { Debug.Log(x); } // block body
```

기초 문법: using System.Linq; = 확장 메서드

List<int>에는 Sum, Where가 **없다**.

그런데 using System.Linq;만 쓰면 사용 가능?

```
1 namespace System.Linq
2 {
3     public static class Enumerable
4     {
5         public static int Sum(this IEnumerable<int> source) { ... }
6     }
7 }
```

핵심은 첫 매개변수의 **this** 키워드.

scores.Sum()이 **마치 List의 메서드처럼** 호출됨.

List<int> 자체는 **건드리지 않았다**.

using으로 “보이게”만 한 **확장 메서드(extension method)**.

실험해 보기 (Lab 5)

- `scores.OrderByDescending(x => x).Take(3)`
→ 상위 3개만 출력
- `scores.Select(x => x * 2)`는 무엇을 만들까?
`.ToList()`로 새 List로 변환
- `List<int>`을 `List<string>`으로 바꿔서
`.OrderBy(s => s.Length)`로 정렬
→ 같은 LINQ가 어떤 타입에든

문제: “매 N초마다”를 자연스럽게

Update에서 타이머를 직접 굴리면...

- `timer -= Time.deltaTime;`
- `if (timer <= 0) { ...; timer = N; }`

동작은 하지만 시간 흐름의 의도가 가려진다.

→ `IEnumerator + yield return`

→ “만든다 → 기다린다 → 다시”를 일직선으로

Lab6_Coroutine.cs

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4
5 public class Lab6_Coroutine : MonoBehaviour
6 {
7     [SerializeField] private float interval = 1f;
8     private Coroutine running;
9
10    void Update()
11    {
12        if (Input.GetKeyDown(KeyCode.C))
13        {
14            if (running == null)
15                running = StartCoroutine(Tick(interval, n => Debug.Log($"Tick #{n}")));
16            else
17                { StopCoroutine(running); running = null; }
18        }
19    }
20
21    IEnumerator Tick(float wait, Action<int> onTick)
22    {
23        int count = 0;
24        while (true)
25        {
26            yield return new WaitForSeconds(wait);
27            count++;
28            onTick(count); // lambda invoked
29        }
30    }
31 }
```

IEnumerator + yield return

yield return X → 중단, X 돌려줌
호출자가 “계속해” → 중단된 곳부터 재개
Unity는 X에 따라 언제 재개할지 결정

n => Debug.Log(\$"Tick #{n}"): 이름 없는 함수

- n => : “n을 받는다”
- 본문: “이걸 한다”
- 함수를 값처럼 인자로 넘기는 사고방식
→ Action<int> 자리에 들어감

Lab 6 Unity 구현 가이드

- 1 Hierarchy → Create Empty → 이름 Lab6
- 2 Lab6_Coroutine.cs 작성, Lab6에 부착
- 3 Inspector의 Interval = 1
→ 0.5나 2로 바꿔 보기 (Play 중에도 가능)
- 4 Play → C 키 (시작/정지 토글)

i 관찰 포인트

Tick이 도는 동안에도
Unity의 다른 Update는 정상 동작.
Coroutine은 양보(yield)와 재개의 반복,
멀티스레드가 아니다.

기초 문법: 콜백 패턴

```
1 IEnumerator Tick(float wait, Action<int> onTick)
2 {
3     ...
4     onTick(count); // call the function we were given
5 }
```

“나중에 호출해 줘”라는 함수 인자 = 콜백(callback)

콜백 분리의 장점: Tick 그대로, “무엇을 할지”만 호출자가 결정.

```
1 StartCoroutine(Tick(1f, n => Debug.Log($"Tick {n}"))); // console
2 StartCoroutine(Tick(1f, n => labelText.text = $" {n}")); // UI
3 StartCoroutine(Tick(1f, n => audio.Play())); // sound
```

Lab 4 event와 같은 책임 분리 사상.

기초 문법: yield return이 돌려주는 “객체” 들

Coroutine 재개 시점은 **돌려주는 객체**로 결정된다.

- `yield return null;`
다음 프레임에 재개. 가장 자주.
- `yield return new WaitForSeconds(t);`
t초 후 재개 (timeScale 영향 받음).
- `yield return new WaitForSecondsRealtime(t);`
실제 시간 t초 후 재개.
- `yield return new WaitUntil(() => 조건);`
조건이 true될 때까지 대기.
- `yield return new WaitForEndOfFrame();`
그 프레임 렌더링 끝난 후.

“편지 내용”에 따라 Unity가 **언제 답장 줄지**를 결정.

실험해 보기 (Lab 6)

- `interval = 0.5f` 또는 `Time.timeScale = 0.1f`
- 콜백을 람다 안에서 분기:
 짝수 → `LogWarning`, 홀수 → `Log`
- `yield return new WaitForSeconds(wait)`을
 `yield return null`로 바꾸면?
 → 매 프레임마다 콜백.
 `yield return`이 무엇을 돌려주느냐가 핵심.

Lab	C# 기능
1	enum + switch 식
2	프로퍼티, =>
3	interface
4	event / Action<T>
5	제네릭 + LINQ + 람다
6	Coroutine + 람다

Unity는 우리에게 무엇을 해 주었나?

6개의 Lab은 모두 **콘솔 C#**으로도 만들 수 있었다.

하지만 Unity 위에서:

- enum이 큐브 **색**으로 바뀌어 보인다
- event가 **색/위치/소리**로 동시 반응
- Coroutine이 **시간을 두고** 펼쳐지는 것을 본다
- 키보드 입력으로 **지금 당장** 실험

★ 핵심 메시지

C#은 본질, Unity는 그것을 잘 보여 주는 무대

Phase 1 & 2

Phase 1 --- 같은 Lab에 한 줄 더

- Lab 1: `Mood.Excited` → 무지개색 큐브
- Lab 2: HP 슬라이더 (Lab 4의 이벤트 패턴 결합)

Phase 2 --- 두 Lab을 합치기

- “작은 인벤토리”: `IItem` + LINQ
(Lab 3 + Lab 5)
- “상태 기계”: `enum` + `event`
(Lab 1 + Lab 4)

Phase 3: 자유 실험

내가 가장 핫걸리는 C# 기능 한 가지를
스스로 Lab으로 만들어 본다.

추천 주제:

- struct vs class: 값 vs 참조
- out/ref 파라미터
- nullable int?와 ??
- abstract class vs interface
- static 멤버
- try/catch/finally

문제 → 콘솔 → Unity 데모 → 실험 과제
본인이 그 흐름을 작성해 본다.

이번 강의의 메시지

- C# 강의의 주인공은 C#
- Unity는 `Console.WriteLine`보다 친절한 무대
- 모든 C# 기능은 불편을 해결하는 도구
외워서 끼우는 것이 아니라 꺼내 쓰는 것

→ 다음 시간 예고

두세 가지 기능을 합쳐
조금 더 큰 Lab으로 발전시키기

**질문은
언제든 환영합니다**